

Motion_Analysis_Assignment

May 23, 2026

1 Motion Analysis Mid-term Assignment

```
[28]: import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from skimage import measure, morphology, io
from skimage.filters import threshold_otsu
```

```
[29]: # Centralised tuning constants used throughout the notebook
NUM_BACKGROUND_FRAMES = 8
BACKGROUND_ALPHA = 0.1
MOTION_THRESHOLD = 0.2
CLEANING_RADIUS_FIXED = 2
CLEANING_RADIUS_OTSU = 3
MIN_BLOB_SIZE = 20
MAX_MATCH_DISTANCE = 40
```

2 Functions definition

2.1 Utility

```
[30]: def load_frames(video_path):
    '''
    Load frames from directory.

    Parameters:
    video_path (str): Path to the directory containing video frames.

    Returns:
    list: A list of frame arrays.
    '''
    video_path = Path(video_path)
    frame_paths = sorted(video_path.glob("*.jpg"))
    if not frame_paths:
        raise ValueError(f"No frames found in {video_path}")

    frames = []
```

```

for frame_path in frame_paths:
    frame = io.imread(str(frame_path))
    if frame.ndim == 3:
        frame = frame.mean(axis=2)  # black and white images
    frames.append(frame.astype(np.float32))

return frames, frame_paths

def visualize_matches(frame_t, frame_t1, blobs_t, blobs_t1, matches,
    title='Blob Matching'):
    """Visualizza i blob nelle due mappe affiancate con centroidi colorati per
    match."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))
    axes[0].imshow(frame_t, cmap='gray')
    axes[0].set_title('Frame t ({} blobs)'.format(blobs_t.max()))
    axes[0].axis('off')
    axes[1].imshow(frame_t1, cmap='gray')
    axes[1].set_title('Frame t+1 ({} blobs)'.format(blobs_t1.max()))
    axes[1].axis('off')
    props_t = {p.label: p for p in measure.regionprops(blobs_t)}
    props_t1 = {p.label: p for p in measure.regionprops(blobs_t1)}
    colors = plt.cm.tab10.colors
    for i, (l_t, l_t1, score) in enumerate(matches):
        color = colors[i % len(colors)]
        if l_t in props_t:
            cy, cx = props_t[l_t].centroid
            axes[0].plot(cx, cy, 'o', color=color, markersize=8)
            axes[0].annotate(str(l_t), (cx, cy), color=color, fontsize=8)
        if l_t1 in props_t1:
            qy, qx = props_t1[l_t1].centroid
            axes[1].plot(qx, qy, 'o', color=color, markersize=8)
            axes[1].annotate(str(l_t1), (qx, qy), color=color, fontsize=8)
    plt.suptitle(title)
    plt.tight_layout()
    plt.show()

def plot_matches(t, rows):
    fig, axes = plt.subplots(len(rows), 2, figsize=(14, 4 * len(rows)))
    if len(rows) == 1:
        axes = np.expand_dims(axes, axis=0)

    colors = plt.cm.tab10.colors

    for row_idx, (title, b_t, b_t1, matches) in enumerate(rows):
        ax_l = axes[row_idx, 0]
        ax_r = axes[row_idx, 1]

```

```

ax_l.imshow(frames[t], cmap='gray')
ax_l.set_title(f"{title} - frame {t}")
ax_l.axis('off')

ax_r.imshow(frames[t + 1], cmap='gray')
ax_r.set_title(f"{title} - frame {t + 1}")
ax_r.axis('off')

props_t = {p.label: p for p in measure.regionprops(b_t)}
props_t1 = {p.label: p for p in measure.regionprops(b_t1)}

for i, (l_t, l_t1, score) in enumerate(matches):
    color = colors[i % len(colors)]
    if l_t in props_t:
        cy, cx = props_t[l_t].centroid
        ax_l.plot(cx, cy, 'o', color=color, markersize=8)
        ax_l.annotate(str(l_t), (cx, cy), color=color, fontsize=8)
    if l_t1 in props_t1:
        qy, qx = props_t1[l_t1].centroid
        ax_r.plot(qx, qy, 'o', color=color, markersize=8)
        ax_r.annotate(str(l_t1), (qx, qy), color=color, fontsize=8)

```

2.2 Background

```

[31]: def initialize_background(frames, num_frames=NUM_BACKGROUND_FRAMES):
    """
    Initialize the background by averaging the first frames in memory.

    Parameters:
    frames (list or array): Sequence of frames already loaded in memory.
    num_frames (int): Number of initial frames to average.

    Returns:
    numpy.ndarray: The estimated background image.
    """
    if len(frames) < num_frames:
        raise ValueError(f"Expected at least {num_frames} frames, found {len(frames)}")

    selected_frames = []
    for frame in frames[:num_frames]:
        if frame is None:
            raise ValueError("Frames must not contain None values")
        if frame.ndim == 3:
            frame = frame.mean(axis=2) # black and white images
        selected_frames.append(frame.astype(np.float32))

```

```

        return np.mean(selected_frames, axis=0)

def update_background(background, new_frame, previous_frame,
    ↪alpha=BACKGROUND_ALPHA, threshold=MOTION_THRESHOLD):
    if background.shape != new_frame.shape:
        raise ValueError("Background and new frame must have the same shape")
    if alpha < 0 or alpha > 1:
        raise ValueError("Alpha must be between 0 and 1")
    if threshold < 0 or threshold > 1:
        raise ValueError("Threshold must be between 0 and 1")
    if np.abs(new_frame - previous_frame).mean() > threshold:
        return background
    return alpha * new_frame + (1 - alpha) * background

```

2.3 Change map

```

[32]: def compute_change_map_fixed_threshold(frame, background, threshold):
    if frame.shape != background.shape:
        raise ValueError("Frame and background must have the same shape")
    if threshold < 0:
        raise ValueError("Threshold must be non-negative")

    frame = frame.astype(np.float32)
    background = background.astype(np.float32)
    difference = np.abs(frame - background)
    return (difference > threshold).astype(np.uint8) * 255

def compute_change_maps_fixed_threshold(frames, initial_background,
    ↪threshold=10, alpha=BACKGROUND_ALPHA, motion_threshold=MOTION_THRESHOLD):
    background = initial_background.astype(np.float32)
    change_maps = []

    for index, frame in enumerate(frames):
        change_map = compute_change_map_fixed_threshold(frame, background,
    ↪threshold)
        change_maps.append(change_map)

    previous_index = max(0, index - 3)
    previous_frame = frames[previous_index]
    background = update_background(
        background,
        frame,
        previous_frame,
        alpha=alpha,
        threshold=motion_threshold,
    )

```

```

    return change_maps

def compute_change_map_otsu(frame, background):
    """
    Compute change map using Otsu's threshold on the difference image.
    Returns the binary map and the computed threshold tau.
    """
    frame = frame.astype(np.float32)
    background = background.astype(np.float32)
    diff = np.abs(frame - background)
    tau = threshold_otsu(diff)
    return (diff > tau).astype(np.uint8) * 255, tau

def compute_change_maps_otsu(frames, initial_background,
    ↪alpha=BACKGROUND_ALPHA, motion_threshold=MOTION_THRESHOLD):
    background = initial_background.astype(np.float32)
    change_maps = []
    thresholds = []
    for index, frame in enumerate(frames):
        change_map, tau = compute_change_map_otsu(frame, background)
        change_maps.append(change_map)

        previous_index = max(0, index - 3)
        previous_frame = frames[previous_index]
        background = update_background(
            background,
            frame,
            previous_frame,
            alpha=alpha,
            threshold=motion_threshold,
        )
        thresholds.append(tau)
    return change_maps, thresholds

def clean_map_fixed(m_t):
    passmask = morphology.disk(CLEANING_RADIUS_FIXED)
    m_t = morphology.closing(m_t > 0, passmask)
    return morphology.opening(m_t > 0, passmask)

def clean_map_otsu(m_t):
    passmask = morphology.disk(CLEANING_RADIUS_OTSU)
    m_t = morphology.closing(m_t > 0, passmask)
    return morphology.opening(m_t > 0, passmask)

```

2.4 Blobs

```
[33]: def compute_blobs(change_map, max_size=MIN_BLOB_SIZE):
    labeled_blobs = measure.label(change_map, connectivity=2)
    cleaned_blobs = morphology.remove_small_objects(labeled_blobs,
    ↪max_size=max_size)
    return cleaned_blobs

def match_blobs_distance(blobs_t, blobs_t_plus_1, max_dist=MAX_MATCH_DISTANCE):
    """Distance-based: abbina ogni blob in t al blob piu vicino in t+1
    usando la distanza euclidea tra centroidi.
    Returns: lista di tuple (label_t, label_t1, distance)
    """
    props_t = measure.regionprops(blobs_t) # calcola le proprietà dei blob in ↪
    ↪t
    props_t1 = measure.regionprops(blobs_t_plus_1)
    if not props_t or not props_t1:
        return []

    matches = []
    for p in props_t:
        cy, cx = p.centroid
        best_label, best_dist = None, float('inf')
        for q in props_t1:
            qy, qx = q.centroid
            d = np.hypot(cy - qy, cx - qx) # distanza euclidea
            if d < best_dist:
                best_dist, best_label = d, q.label
        if best_dist < max_dist: #
            matches.append((p.label, best_label, best_dist))
    return matches

def match_blobs_descriptor(blobs_t, blobs_t_plus_1, frame_t, frame_t1,
    max_dist=MAX_MATCH_DISTANCE, w_pos=1.0, w_area=0.5, ↪
    ↪w_intensity=0.3):
    """Descriptor-based: combina distanza tra centroidi con similarita
    di area e intensita media del blob nell'immagine originale.

    score = w_pos * dist_centroid
    + w_area * |area_t - area_t1| / max(area_t, area_t1)
    + w_intensity * |mean_int_t - mean_int_t1| / 255

    Returns: lista di tuple (label_t, label_t1, score)
    """
    props_t = measure.regionprops(blobs_t, intensity_image=frame_t.astype(np.
    ↪float32))
```

```

    props_t1 = measure.regionprops(blobs_t_plus_1, intensity_image=frame_t1.
↪astype(np.float32))
    if not props_t or not props_t1:
        return []
    matches = []
    for p in props_t:
        cy, cx = p.centroid
        best_label, best_score = None, float('inf')
        for q in props_t1:
            qy, qx = q.centroid
            dist = np.hypot(cy - qy, cx - qx)
            if dist >= max_dist:
                continue
            area_sim = abs(p.area - q.area) / max(p.area, q.area)
            int_sim = abs(p.mean_intensity - q.mean_intensity) / 255.0
            score = w_pos * dist + w_area * area_sim + w_intensity * int_sim
            if score < best_score:
                best_score, best_label = score, q.label
        if best_label is not None:
            matches.append((p.label, best_label, best_score))
    return matches

```

3 Background initialization

First we initialize the background image by taking the average of the first 8 frames and show the result.

```

[34]: video_dir = Path("video/station")
frames, frame_paths = load_frames(video_dir)
initial_background = initialize_background(frames, ↵
↪num_frames=NUM_BACKGROUND_FRAMES)

fig, ax = plt.subplots(figsize=(7, 5))
ax.imshow(initial_background, cmap="gray",
          vmin=initial_background.min(), vmax=initial_background.max())
ax.set_title("Initial background B0")
ax.axis("off")
plt.tight_layout()
plt.show()

```

Initial background B0



4 Change detection maps

The change detection maps highlight pixels whose intensity differs significantly from the background estimate. We will focus on two factors that will determine their accuracy:

- **Background estimation method:** different strategies (e.g., a static average, a running average with selective updating, or more advanced models) produce different reference images B , which change the distribution of the difference image $|I_t - B|$ and therefore the detected regions.
- **Threshold selection (τ):** the value used to binarize $|I_t - B|$ determines sensitivity vs. false positives. A low τ increases detection of small changes (more false positives); a high τ suppresses small motion (more false negatives).

For this assignment we will:

- Explore one background update technique (running average with a freeze condition to avoid absorbing moving objects), and
- Compare two threshold-selection methods: a fixed heuristic threshold and Otsu's automatic thresholding.

We will visually and quantitatively compare results to show how background choice and `interact`.

4.1 Fixed threshold

To choose a fixed threshold we compute and print the intensity histogram for the middle frame.

```
[35]: # Inspect the difference-image histogram to choose a heuristic threshold
sample_index = len(frames) // 2
sample_frame = frames[sample_index].astype(np.float32)
difference_image = np.abs(sample_frame - initial_background.astype(np.float32))

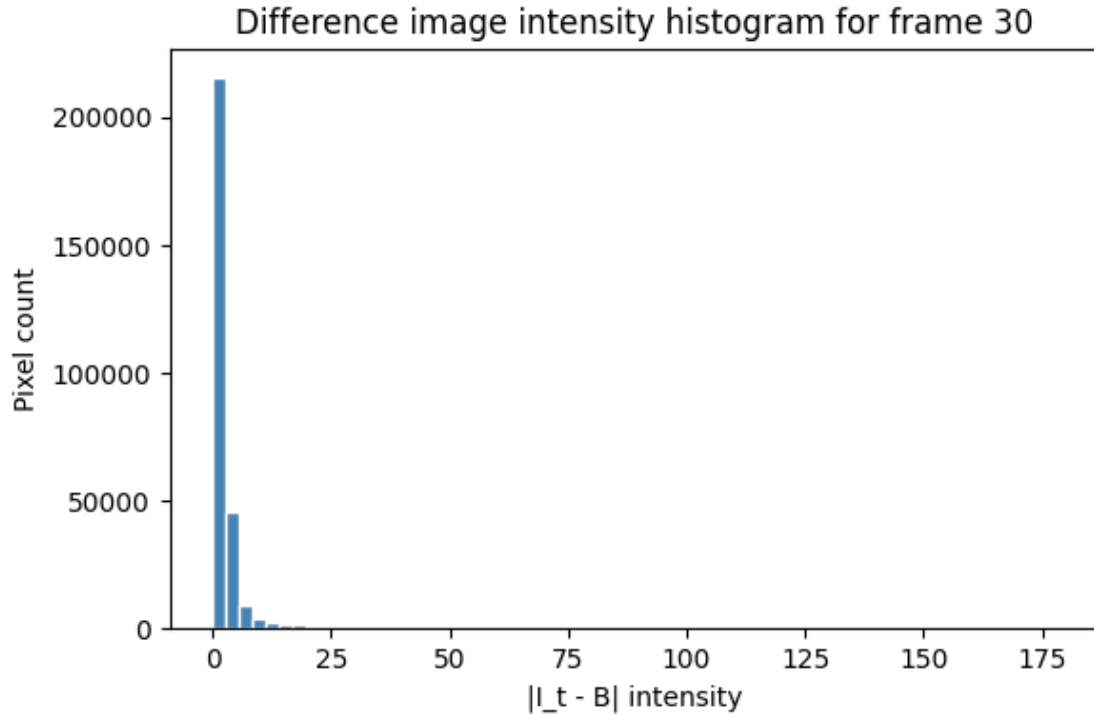
hist, bin_edges = np.histogram(difference_image.ravel(), bins=64, range=(0,
    ↪ difference_image.max()))

print(f"Histogram for |I_t - B| at frame {sample_index}")
for left, right, count in zip(bin_edges[:-1], bin_edges[1:], hist):
    bar = "#" * int(40 * count / hist.max()) if hist.max() else ""
    print(f"{left:7.2f} - {right:7.2f} | {count:8d} {bar}")

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(difference_image.ravel(), bins=64, color="steelblue", edgecolor="white")
ax.set_title(f"Difference image intensity histogram for frame {sample_index}")
ax.set_xlabel("|I_t - B| intensity")
ax.set_ylabel("Pixel count")
plt.tight_layout()
plt.show()
```

```
Histogram for |I_t - B| at frame 30
 0.00 - 2.79 | 215765 #####
 2.79 - 5.57 | 45680 #####
 5.57 - 8.36 | 8813 #
 8.36 - 11.14 | 3633
11.14 - 13.93 | 2493
13.93 - 16.71 | 1731
16.71 - 19.50 | 1326
19.50 - 22.29 | 1105
22.29 - 25.07 | 1004
25.07 - 27.86 | 822
27.86 - 30.64 | 805
30.64 - 33.43 | 722
33.43 - 36.22 | 742
36.22 - 39.00 | 715
39.00 - 41.79 | 712
41.79 - 44.57 | 654
44.57 - 47.36 | 715
47.36 - 50.14 | 745
50.14 - 52.93 | 717
52.93 - 55.72 | 603
```

55.72	-	58.50		545
58.50	-	61.29		560
61.29	-	64.07		522
64.07	-	66.86		459
66.86	-	69.65		372
69.65	-	72.43		346
72.43	-	75.22		380
75.22	-	78.00		347
78.00	-	80.79		355
80.79	-	83.57		395
83.57	-	86.36		397
86.36	-	89.15		471
89.15	-	91.93		555
91.93	-	94.72		611
94.72	-	97.50		466
97.50	-	100.29		401
100.29	-	103.07		345
103.07	-	105.86		380
105.86	-	108.65		359
108.65	-	111.43		388
111.43	-	114.22		404
114.22	-	117.00		345
117.00	-	119.79		408
119.79	-	122.58		432
122.58	-	125.36		480
125.36	-	128.15		531
128.15	-	130.93		380
130.93	-	133.72		401
133.72	-	136.50		368
136.50	-	139.29		375
139.29	-	142.08		288
142.08	-	144.86		312
144.86	-	147.65		429
147.65	-	150.43		591
150.43	-	153.22		818
153.22	-	156.01		708
156.01	-	158.79		461
158.79	-	161.58		416
161.58	-	164.36		459
164.36	-	167.15		269
167.15	-	169.93		112
169.93	-	172.72		38
172.72	-	175.51		12
175.51	-	178.29		7



```
[36]: FIXED_THRESHOLD = 20

change_maps_fixed_threshold = compute_change_maps_fixed_threshold(
    frames,
    initial_background,
    threshold=FIXED_THRESHOLD,
    alpha=0.1,
    motion_threshold=0.2
)
```

Looking at the histogram's tail we evaluate that $t_f = 20$ is a good threshold.

4.2 Otsu's Method

Otsu's method is an automatic thresholding technique that computes a threshold value that minimizes the within-class variance of the resulting binary image. Instead of manually choosing a fixed threshold, Otsu's algorithm exhaustively searches over all possible threshold values and picks the one that best separates the histogram into two classes (background and foreground).

We print the min, max and mean threshold values found by Otsu's method.

```
[37]: change_maps_otsu, otsu_thresholds = compute_change_maps_otsu(frames,
    ↪ initial_background)
```

```
print("Otsu threshold - min: {:.1f} max: {:.1f} mean: {:.1f}".format(
    min(otsu_thresholds), max(otsu_thresholds), sum(otsu_thresholds)/
    len(otsu_thresholds)))
```

Otsu threshold - min: 22.8 max: 72.6 mean: 55.7

4.3 Change maps: Otsu vs Fixed

Now let's compute the change maps and visualize a set of frames {10, 35, 55} to observe the differences:

```
[38]: # Visual comparison: fixed threshold vs Otsu on frames 10, 35, and 55
indices = [10, 35, 55]
valid_indices = [idx for idx in indices if 0 <= idx < len(frames)]

if len(valid_indices) < len(indices):
    missing = [idx for idx in indices if idx not in valid_indices]
    print(f"Skipped out-of-range frame indices: {missing}")

fig, axes = plt.subplots(len(valid_indices), 3, figsize=(15, 4 *
    len(valid_indices)))
if len(valid_indices) == 1:
    axes = np.expand_dims(axes, axis=0)

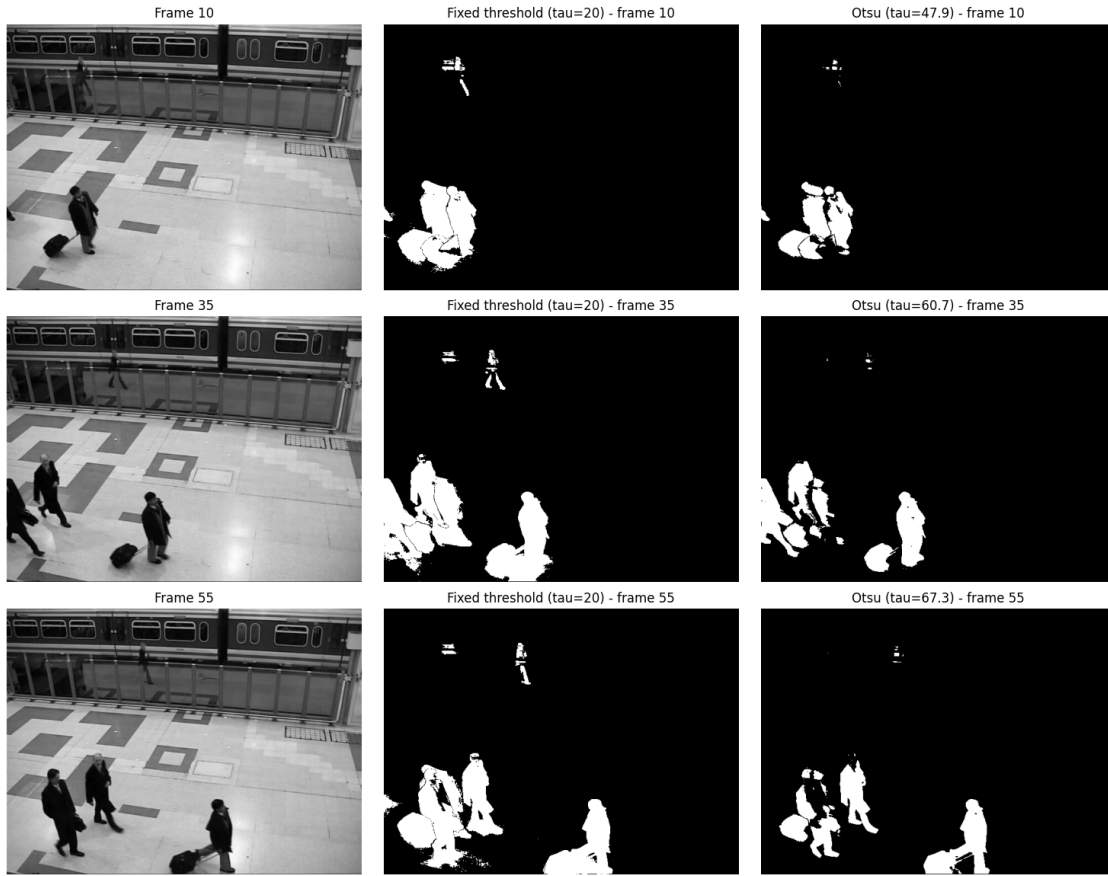
for row, idx in enumerate(valid_indices):
    axes[row, 0].imshow(frames[idx], cmap="gray")
    axes[row, 0].set_title(f"Frame {idx}")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(change_maps_fixed_threshold[idx], cmap="gray", vmin=0,
    vmax=255)
    axes[row, 1].set_title(f"Fixed threshold (tau={FIXED_THRESHOLD}) - frame_
    {idx}")
    axes[row, 1].axis("off")

    axes[row, 2].imshow(change_maps_otsu[idx], cmap="gray", vmin=0, vmax=255)
    axes[row, 2].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - frame_
    {idx}")
    axes[row, 2].axis("off")

plt.suptitle("Change maps comparison: fixed threshold vs Otsu", fontsize=18,
    y=1.02)
plt.tight_layout(h_pad=2)
plt.show()
```

Change maps comparison: fixed threshold vs Otsu



4.4 Comment

Across these three frames, the fixed-threshold maps are generally more prone to false positive, the shadow of the pedestrian that enter the scene are detected as foreground objects. In contrast, Otsu adapts the threshold per frame by increasing the threshold as more pedestrian enter in scene, this create a change map that cleary separates the moving objects from the background, but at the cost of slightly more fragmented silhouettes of the people in scene. We can also notice that with Otsu the person more far in the scene is completely cut.

5 Map Cleaning

Segmentation quality can be improved with appropriate post-processing. Binary masks are cleaned using morphological operations (opening and closing) to remove small noise, fill small holes, and connect nearby fragments.

```

[39]: # Clean the change maps using morphological opening
cleaned_fixed_threshold = [clean_map_fixed(m) for m in change_maps_fixed_threshold]
cleaned_otsu = [clean_map_otsu(m) for m in change_maps_otsu]

indices = [10, 35, 55]
valid_indices = [idx for idx in indices if 0 <= idx < len(frames)]

fig, axes = plt.subplots(len(valid_indices), 4, figsize=(16, 4 * len(valid_indices)))
if len(valid_indices) == 1:
    axes = np.expand_dims(axes, axis=0)

for row, idx in enumerate(valid_indices):
    axes[row, 0].imshow(change_maps_fixed_threshold[idx], cmap="gray", vmin=0, vmax=255)
    axes[row, 0].set_title(f"Fixed (tau={FIXED_THRESHOLD}) - raw")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(cleaned_fixed_threshold[idx], cmap="gray")
    axes[row, 1].set_title(f"Fixed (tau={FIXED_THRESHOLD}) - cleaned")
    axes[row, 1].axis("off")

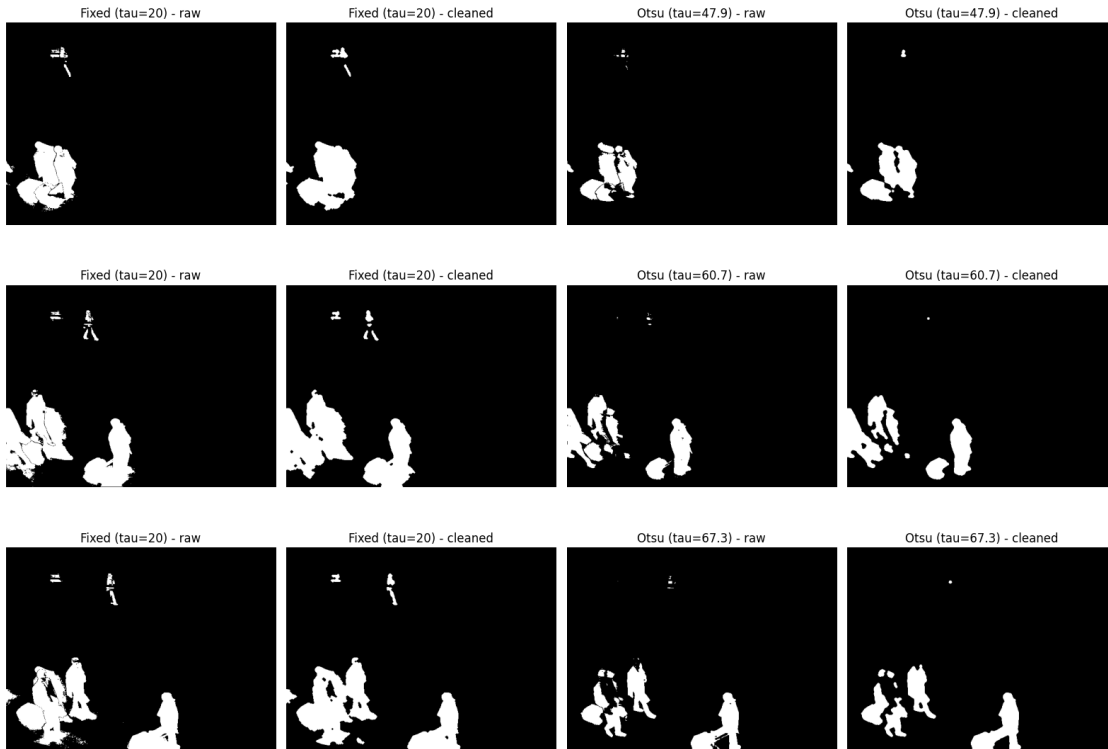
    axes[row, 2].imshow(change_maps_otsu[idx], cmap="gray")
    axes[row, 2].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - raw")
    axes[row, 2].axis("off")

    axes[row, 3].imshow(cleaned_otsu[idx], cmap="gray")
    axes[row, 3].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - cleaned")
    axes[row, 3].axis("off")

plt.suptitle("Map cleaning effect: raw vs cleaned maps", fontsize=18, y=1.02)
plt.tight_layout(h_pad=2)
plt.show()

```

Map cleaning effect: raw vs cleaned maps



5.1 Comment

After cleaning, the binary masks are more uniform and better suited for blob detection and matching, which reduces spurious connected components and improves the reliability of subsequent motion analysis.

6 Blobs

A “blob” (Binary Large Object) is a contiguous group of connected pixels in an image that share a common property. Computing blobs relies on an operation called Connected Component Labeling, this algorithm scans the binary image, groups adjacent active pixels together, and assigns a unique integer ID to each distinct cluster, for this operation we rely on the skimage library. Once labeled, we are able to treat these clusters as individual entities, allowing us to extract mathematical properties like area, shape, or a central coordinate (centroid) to identify and track distinct objects moving through a scene.

```
[40]: t = 40

clean_map_fixed_t = clean_map_fixed(change_maps_fixed_threshold[t])
```

```

clean_map_fixed_t1 = clean_map_fixed(change_maps_fixed_threshold[t + 1])

clean_map_otsu_t = clean_map_otsu(change_maps_otsu[t])
clean_map_otsu_t1 = clean_map_otsu(change_maps_otsu[t + 1])

blobs_fixed_t = compute_blobs(clean_map_fixed_t)
blobs_otsu_t = compute_blobs(clean_map_otsu_t)
blobs_fixed_t1 = compute_blobs(clean_map_fixed_t1)
blobs_otsu_t1 = compute_blobs(clean_map_otsu_t1)

print("Blob_t_fixed")
print('Frame t={}: {} blobs'.format(t, blobs_fixed_t.max()))
print('Frame t+1={}: {} blobs'.format(t + 1, blobs_fixed_t1.max()))

print("\nBlob_t_otsu")
print('Frame t={}: {} blobs'.format(t, blobs_otsu_t.max()))
print('Frame t+1={}: {} blobs'.format(t + 1, blobs_otsu_t1.max()))

```

```

Blob_t_fixed
Frame t=40: 6 blobs
Frame t+1=41: 9 blobs

```

```

Blob_t_otsu
Frame t=40: 6 blobs
Frame t+1=41: 6 blobs

```

6.1 Blobs matching

Blob matching links connected components between consecutive frames to form short tracks. The notebook provides two complementary strategies implemented by `match_blobs_distance` and `match_blobs_descriptor`.

- **Distance-based (`match_blobs_distance`):** matches each blob in frame t to the nearest blob in frame $t+1$ using the Euclidean distance between centroids. The function returns a list of tuples (`label_t`, `label_t1`, `distance`). Pros: simple and fast, effective when objects are well separated and inter-frame displacements are small. Cons: ambiguous when multiple candidates are at similar distances, and fragile under rapid motion or occlusions; it relies on a `max_dist` threshold to reject unlikely matches.
- **Descriptor-based (`match_blobs_descriptor`):** computes a composite score that combines centroid distance with appearance/shape descriptors (area and mean intensity) using weights `w_pos`, `w_area`, and `w_intensity`. The function returns tuples (`label_t`, `label_t1`, `score`) where a lower score indicates a better match. Pros: reduces assignment ambiguity by preferring matches that are similar in size and appearance, making it more robust when objects cross or have different sizes. Cons: more computationally expensive and requires tuning of the weighting parameters; still limited under severe occlusion or large scale changes.

```

[41]: t = 40

# Recompute cleaned maps and blobs for the selected pair
clean_map_fixed_t = clean_map_fixed(change_maps_fixed_threshold[t])
clean_map_fixed_t1 = clean_map_fixed(change_maps_fixed_threshold[t + 1])
clean_map_otsu_t = clean_map_otsu(change_maps_otsu[t])
clean_map_otsu_t1 = clean_map_otsu(change_maps_otsu[t + 1])

blobs_fixed_t = compute_blobs(clean_map_fixed_t)
blobs_fixed_t1 = compute_blobs(clean_map_fixed_t1)
blobs_otsu_t = compute_blobs(clean_map_otsu_t)
blobs_otsu_t1 = compute_blobs(clean_map_otsu_t1)

# Compute matches
matches_dist_fixed = match_blobs_distance(blobs_fixed_t, blobs_fixed_t1,
    ↪max_dist=MAX_MATCH_DISTANCE)
matches_dist_otsu = match_blobs_distance(blobs_otsu_t, blobs_otsu_t1,
    ↪max_dist=MAX_MATCH_DISTANCE)
matches_desc_fixed = match_blobs_descriptor(blobs_fixed_t, blobs_fixed_t1,
    ↪frames[t], frames[t + 1], max_dist=MAX_MATCH_DISTANCE)
matches_desc_otsu = match_blobs_descriptor(blobs_otsu_t, blobs_otsu_t1,
    ↪frames[t], frames[t + 1], max_dist=MAX_MATCH_DISTANCE)

# Prepare rows: (title, blobs_t, blobs_t1, matches)
rows = [
    (f"Fixed - distance-based (max_dist={MAX_MATCH_DISTANCE})", blobs_fixed_t,
    ↪blobs_fixed_t1, matches_dist_fixed),
    (f"Otsu - distance-based (max_dist={MAX_MATCH_DISTANCE})", blobs_otsu_t,
    ↪blobs_otsu_t1, matches_dist_otsu),
    ("Fixed - descriptor-based", blobs_fixed_t, blobs_fixed_t1,
    ↪matches_desc_fixed),
    ("Otsu - descriptor-based", blobs_otsu_t, blobs_otsu_t1, matches_desc_otsu),
]

plot_matches(t, rows)

plt.suptitle('Blob matching: Distance vs Descriptor (Fixed vs Otsu)',
    ↪fontsize=16, y=0.98)
plt.tight_layout()
plt.show()

# Print short summaries
print('Fixed distance-based matches: {}'.format(len(matches_dist_fixed)))
print('Otsu distance-based matches: {}'.format(len(matches_dist_otsu)))
print('Fixed descriptor-based matches: {}'.format(len(matches_desc_fixed)))
print('Otsu descriptor-based matches: {}'.format(len(matches_desc_otsu)))

```

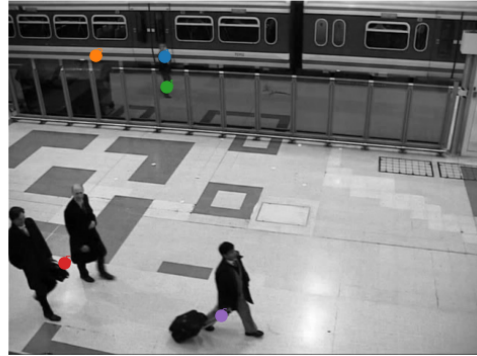
```
C:\Users\chiol\AppData\Local\Temp\ipykernel_15372\529160326.py:54:
FutureWarning: `RegionProperties.mean_intensity` is deprecated starting in
version 0.26 and will be removed in version 2.0. Use
`RegionProperties.intensity_mean` instead.
    int_sim = abs(p.mean_intensity - q.mean_intensity) / 255.0
```

Blob matching: Distance vs Descriptor (Fixed vs Otsu)

Fixed — distance-based (max_dist=40) — frame 40



Fixed — distance-based (max_dist=40) — frame 41



Otsu — distance-based (max_dist=40) — frame 40



Otsu — distance-based (max_dist=40) — frame 41



Fixed — descriptor-based — frame 40



Fixed — descriptor-based — frame 41



Otsu — descriptor-based — frame 40



Otsu — descriptor-based — frame 41



Fixed distance-based matches: 5
Otsu distance-based matches: 6
Fixed descriptor-based matches: 5
Otsu descriptor-based matches: 6

6.2 Results summary

- **Fixed vs Otsu:** Fixed-threshold maps tend to produce larger, noisier blobs (more false positives and merged components). Otsu's adaptive threshold reduces background noise but can fragment distant or low-contrast objects.
- **Distance-based vs Descriptor-based matching:** Distance matching recovers the obvious nearest-centroid associations and is fast, but it becomes ambiguous when multiple candidates are similarly close or when motion is large. Descriptor-based matching (area + mean intensity) reduces these assignment errors by preferring appearance/shape-consistent matches at the cost of additional computation and parameter tuning.
- **Observed outcome (this example):** Without a more deep analysis is not possible to determine which matching and threshold strategy perform better, in every combination there are evidence of false positive. Otsu seems more consistent in tracking the pedestrian more far away in the scene, but tends to detect as a blob the shadow of the people in the scene. The fixed threshold seems to be unable to distinguish between the two people who enter later in the scene and walk close to each other treating them like a single object.

6.3 Another case

We study another video that is a perfect example of lighting changes. In the video a guy moves from a dim light place to a brighter environment.

```
[42]: video_dir = Path("video/david")
frames, frame_paths = load_frames(video_dir)
initial_background = initialize_background(frames, num_frames=10)

fig, ax = plt.subplots(figsize=(7, 5))
ax.imshow(initial_background, cmap="gray",
          vmin=initial_background.min(), vmax=initial_background.max())
ax.set_title("Initial background B0")
ax.axis("off")
plt.tight_layout()
plt.show()
```

Initial background B0



```
[43]: # Inspect the difference-image histogram to choose a heuristic threshold
sample_index = len(frames) // 2
sample_frame = frames[sample_index].astype(np.float32)
difference_image = np.abs(sample_frame - initial_background.astype(np.float32))

hist, bin_edges = np.histogram(difference_image.ravel(), bins=64, range=(0,
↳ difference_image.max()))

print(f"Histogram for |I_t - B| at frame {sample_index}")
for left, right, count in zip(bin_edges[:-1], bin_edges[1:], hist):
    bar = "#" * int(40 * count / hist.max()) if hist.max() else ""
    print(f"{left:7.2f} - {right:7.2f} | {count:8d} {bar}")

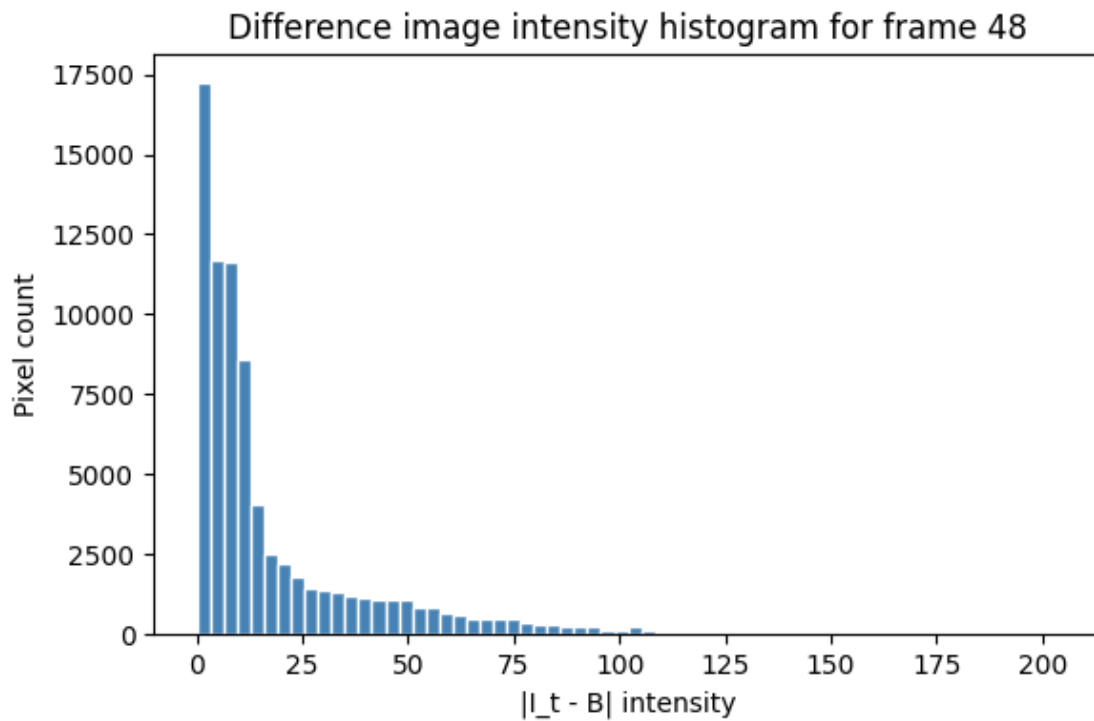
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(difference_image.ravel(), bins=64, color="steelblue", edgecolor="white")
ax.set_title(f"Difference image intensity histogram for frame {sample_index}")
ax.set_xlabel("|I_t - B| intensity")
ax.set_ylabel("Pixel count")
plt.tight_layout()
```

```
plt.show()
```

Histogram for $|I_t - B|$ at frame 48

0.00 -	3.19	17250	#####
3.19 -	6.37	11692	#####
6.37 -	9.56	11635	#####
9.56 -	12.75	8576	#####
12.75 -	15.93	4038	#####
15.93 -	19.12	2534	#####
19.12 -	22.31	2223	#####
22.31 -	25.49	1802	####
25.49 -	28.68	1460	###
28.68 -	31.86	1384	###
31.86 -	35.05	1322	###
35.05 -	38.24	1194	##
38.24 -	41.42	1132	##
41.42 -	44.61	1112	##
44.61 -	47.80	1076	##
47.80 -	50.98	1096	##
50.98 -	54.17	870	##
54.17 -	57.36	869	##
57.36 -	60.54	648	#
60.54 -	63.73	587	#
63.73 -	66.92	494	#
66.92 -	70.10	479	#
70.10 -	73.29	466	#
73.29 -	76.47	465	#
76.47 -	79.66	357	
79.66 -	82.85	334	
82.85 -	86.03	303	
86.03 -	89.22	234	
89.22 -	92.41	274	
92.41 -	95.59	252	
95.59 -	98.78	150	
98.78 -	101.97	148	
101.97 -	105.15	231	
105.15 -	108.34	109	
108.34 -	111.53	2	
111.53 -	114.71	0	
114.71 -	117.90	0	
117.90 -	121.09	0	
121.09 -	124.27	0	
124.27 -	127.46	0	
127.46 -	130.64	0	
130.64 -	133.83	0	
133.83 -	137.02	0	
137.02 -	140.20	0	
140.20 -	143.39	0	

143.39 - 146.58		0
146.58 - 149.76		0
149.76 - 152.95		0
152.95 - 156.14		0
156.14 - 159.32		0
159.32 - 162.51		0
162.51 - 165.70		0
165.70 - 168.88		0
168.88 - 172.07		0
172.07 - 175.26		1
175.26 - 178.44		0
178.44 - 181.63		0
181.63 - 184.81		0
184.81 - 188.00		0
188.00 - 191.19		0
191.19 - 194.37		0
194.37 - 197.56		0
197.56 - 200.75		0
200.75 - 203.93		1



```
[44]: FIXED_THRESHOLD = 50

change_maps_fixed_threshold = compute_change_maps_fixed_threshold(
```

```

frames,
initial_background,
threshold=FIXED_THRESHOLD,
alpha=0.1,
motion_threshold=0.2
)

change_maps_otu, otsu_thresholds = compute_change_maps_otu(frames,
↪initial_background)

print("Otsu threshold - min: {:.1f} max: {:.1f} mean: {:.1f}".format(
    min(otsu_thresholds), max(otsu_thresholds), sum(otsu_thresholds)/
↪len(otsu_thresholds)))

```

Otsu threshold - min: 7.6 max: 90.0 mean: 42.9

```

[45]: indices = [20, 55, 95]
valid_indices = [idx for idx in indices if 0 <= idx < len(frames)]

if len(valid_indices) < len(indices):
    missing = [idx for idx in indices if idx not in valid_indices]
    print(f"Skipped out-of-range frame indices: {missing}")

fig, axes = plt.subplots(len(valid_indices), 3, figsize=(15, 4 *
↪len(valid_indices)))
if len(valid_indices) == 1:
    axes = np.expand_dims(axes, axis=0)

for row, idx in enumerate(valid_indices):
    axes[row, 0].imshow(frames[idx], cmap="gray")
    axes[row, 0].set_title(f"Frame {idx}")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(change_maps_fixed_threshold[idx], cmap="gray", vmin=0,
↪vmax=255)
    axes[row, 1].set_title(f"Fixed threshold (tau={FIXED_THRESHOLD}) - frame
↪{idx}")
    axes[row, 1].axis("off")

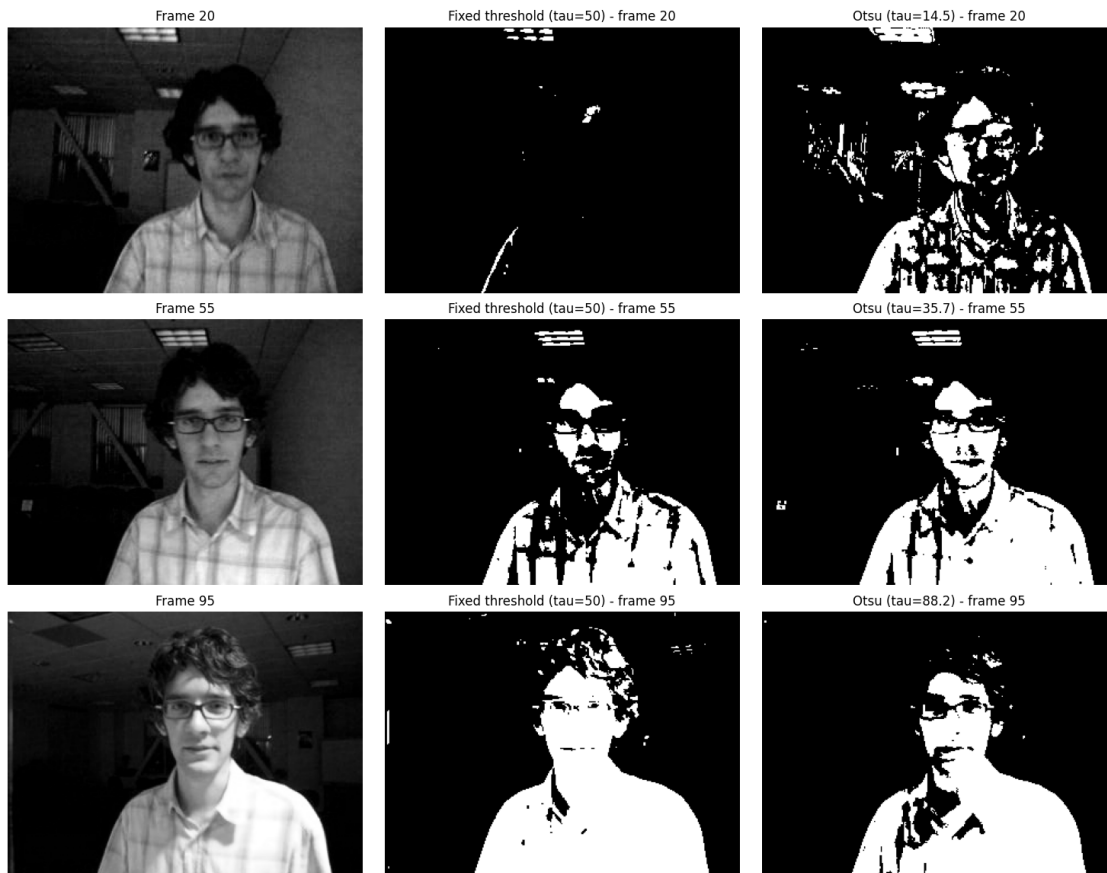
    axes[row, 2].imshow(change_maps_otu[idx], cmap="gray", vmin=0, vmax=255)
    axes[row, 2].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - frame
↪{idx}")
    axes[row, 2].axis("off")

plt.suptitle("Change maps comparison: fixed threshold vs Otsu", fontsize=18,
↪y=1.02)
plt.tight_layout(h_pad=2)

```

```
plt.show()
```

Change maps comparison: fixed threshold vs Otsu



```
[46]: # Clean the change maps using morphological opening
cleaned_fixed_threshold = [clean_map_fixed(m) for m in
    ↪ change_maps_fixed_threshold]
cleaned_otu = [clean_map_otu(m) for m in change_maps_otu]

indices = [20, 55, 95]
valid_indices = [idx for idx in indices if 0 <= idx < len(frames)]

fig, axes = plt.subplots(len(valid_indices), 4, figsize=(16, 4 *
    ↪ len(valid_indices)))
if len(valid_indices) == 1:
    axes = np.expand_dims(axes, axis=0)

for row, idx in enumerate(valid_indices):
    axes[row, 0].imshow(change_maps_fixed_threshold[idx], cmap="gray", vmin=0,
    ↪ vmax=255)
```

```

axes[row, 0].set_title(f"Fixed (tau={FIXED_THRESHOLD}) - raw")
axes[row, 0].axis("off")

axes[row, 1].imshow(cleaned_fixed_threshold[idx], cmap="gray")
axes[row, 1].set_title(f"Fixed (tau={FIXED_THRESHOLD}) - cleaned")
axes[row, 1].axis("off")

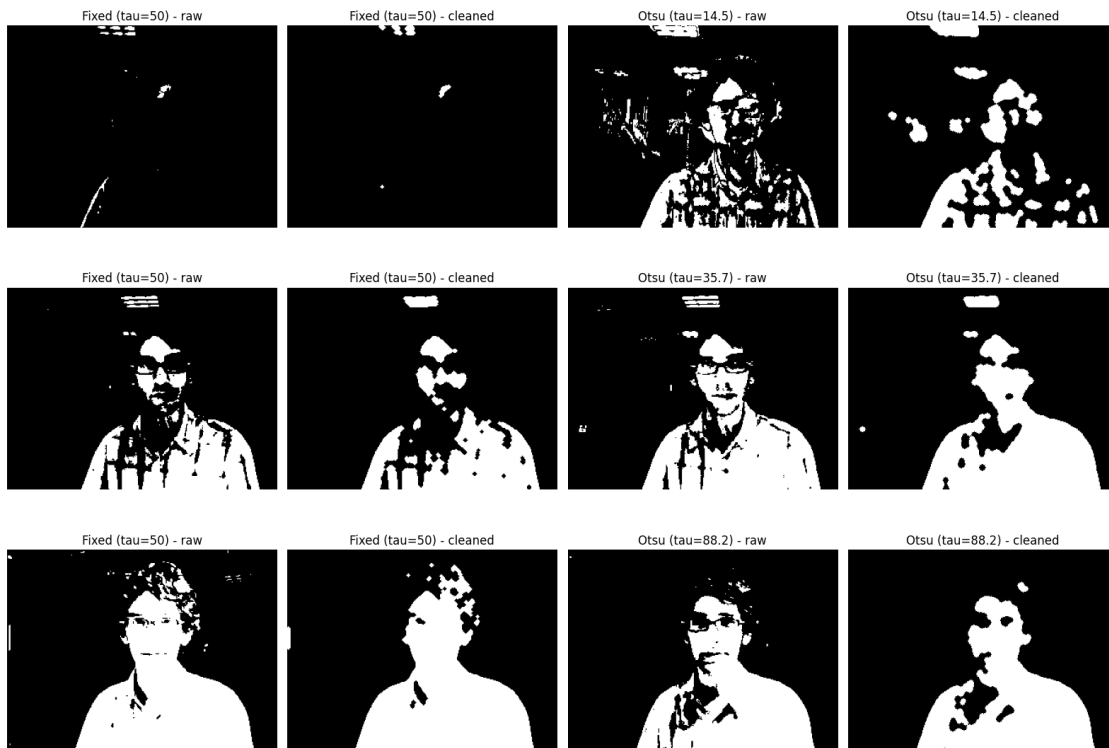
axes[row, 2].imshow(change_maps_otsu[idx], cmap="gray")
axes[row, 2].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - raw")
axes[row, 2].axis("off")

axes[row, 3].imshow(cleaned_otsu[idx], cmap="gray")
axes[row, 3].set_title(f"Otsu (tau={otsu_thresholds[idx]:.1f}) - cleaned")
axes[row, 3].axis("off")

plt.suptitle("Map cleaning effect: raw vs cleaned maps", fontsize=18, y=1.02)
plt.tight_layout(h_pad=2)
plt.show()

```

Map cleaning effect: raw vs cleaned maps



```
[47]: t = len(frames) // 2

clean_map_fixed_t = clean_map_fixed(change_maps_fixed_threshold[t])
clean_map_fixed_t1 = clean_map_fixed(change_maps_fixed_threshold[t + 1])

clean_map_otsu_t = clean_map_otsu(change_maps_otsu[t])
clean_map_otsu_t1 = clean_map_otsu(change_maps_otsu[t + 1])

blobs_fixed_t = compute_blobs(clean_map_fixed_t, max_size=200)
blobs_otsu_t = compute_blobs(clean_map_otsu_t, max_size=200)
blobs_fixed_t1 = compute_blobs(clean_map_fixed_t1, max_size=200)
blobs_otsu_t1 = compute_blobs(clean_map_otsu_t1, max_size=200)

print("Blob_t_fixed")
print('Frame t={}: {} blobs'.format(t, blobs_fixed_t.max()))
print('Frame t+1={}: {} blobs'.format(t + 1, blobs_fixed_t1.max()))

print("\nBlob_t_otsu")
print('Frame t={}: {} blobs'.format(t, blobs_otsu_t.max()))
print('Frame t+1={}: {} blobs'.format(t + 1, blobs_otsu_t1.max()))
```

```
Blob_t_fixed
Frame t=48: 12 blobs
Frame t+1=49: 14 blobs
```

```
Blob_t_otsu
Frame t=48: 4 blobs
Frame t+1=49: 4 blobs
```

```
[48]: t = len(frames) // 2

# Recompute cleaned maps and blobs for the selected pair
clean_map_fixed_t = clean_map_fixed(change_maps_fixed_threshold[t])
clean_map_fixed_t1 = clean_map_fixed(change_maps_fixed_threshold[t + 1])
clean_map_otsu_t = clean_map_otsu(change_maps_otsu[t])
clean_map_otsu_t1 = clean_map_otsu(change_maps_otsu[t + 1])

blobs_fixed_t = compute_blobs(clean_map_fixed_t)
blobs_fixed_t1 = compute_blobs(clean_map_fixed_t1)
blobs_otsu_t = compute_blobs(clean_map_otsu_t)
blobs_otsu_t1 = compute_blobs(clean_map_otsu_t1)

# Compute matches
matches_dist_fixed = match_blobs_distance(blobs_fixed_t, blobs_fixed_t1,
    ↪max_dist=MAX_MATCH_DISTANCE)
matches_dist_otsu = match_blobs_distance(blobs_otsu_t, blobs_otsu_t1,
    ↪max_dist=MAX_MATCH_DISTANCE)
```

```

matches_desc_fixed = match_blobs_descriptor(blobs_fixed_t, blobs_fixed_t1,
↳frames[t], frames[t + 1], max_dist=MAX_MATCH_DISTANCE)
matches_desc_otsu = match_blobs_descriptor(blobs_otsu_t, blobs_otsu_t1,
↳frames[t], frames[t + 1], max_dist=MAX_MATCH_DISTANCE)

# Prepare rows: (title, blobs_t, blobs_t1, matches)
rows = [
    (f"Fixed - distance-based (max_dist={MAX_MATCH_DISTANCE})", blobs_fixed_t,
↳blobs_fixed_t1, matches_dist_fixed),
    (f"Otsu - distance-based (max_dist={MAX_MATCH_DISTANCE})", blobs_otsu_t,
↳blobs_otsu_t1, matches_dist_otsu),
    ("Fixed - descriptor-based", blobs_fixed_t, blobs_fixed_t1,
↳matches_desc_fixed),
    ("Otsu - descriptor-based", blobs_otsu_t, blobs_otsu_t1, matches_desc_otsu),
]

plot_matches(t, rows)

plt.suptitle('Blob matching: Distance vs Descriptor (Fixed vs Otsu)',
↳fontsize=16, y=0.98)
plt.tight_layout()
plt.show()

# Print short summaries
print('Fixed distance-based matches: {}'.format(len(matches_dist_fixed)))
print('Otsu distance-based matches: {}'.format(len(matches_dist_otsu)))
print('Fixed descriptor-based matches: {}'.format(len(matches_desc_fixed)))
print('Otsu descriptor-based matches: {}'.format(len(matches_desc_otsu)))

```

C:\Users\chiol\AppData\Local\Temp\ipykernel_15372\529160326.py:54:
FutureWarning: `RegionProperties.mean\_intensity` is deprecated starting in
version 0.26 and will be removed in version 2.0. Use
`RegionProperties.intensity\_mean` instead.

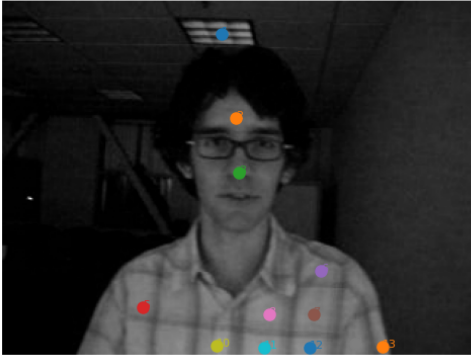
```

int_sim = abs(p.mean_intensity - q.mean_intensity) / 255.0

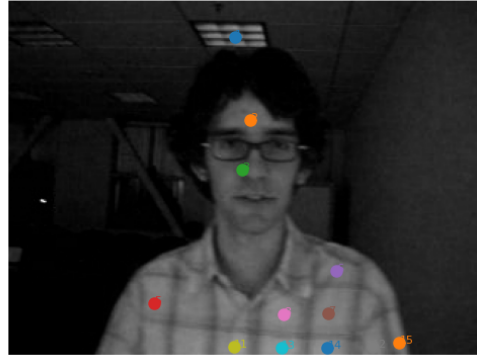
```

Blob matching: Distance vs Descriptor (Fixed vs Otsu)

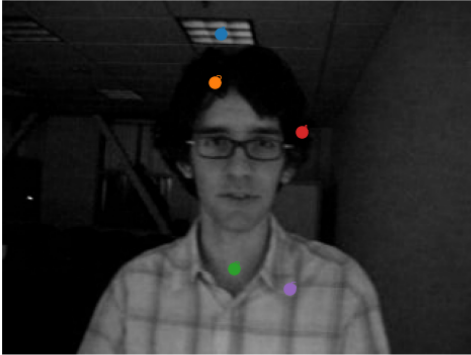
Fixed — distance-based (max_dist=40) — frame 48



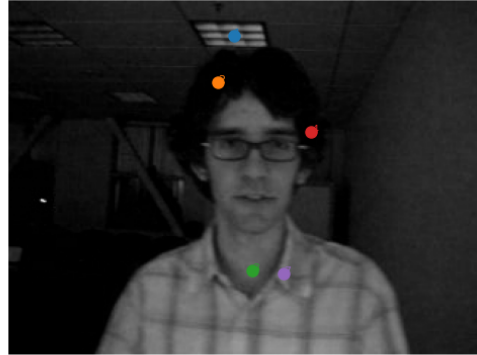
Fixed — distance-based (max_dist=40) — frame 49



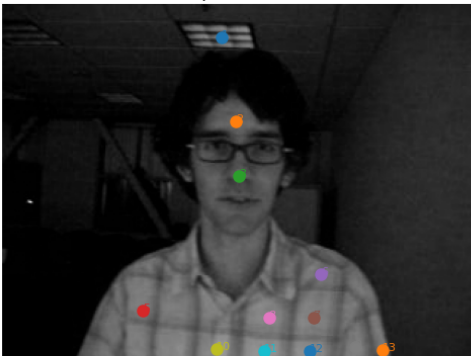
Otsu — distance-based (max_dist=40) — frame 48



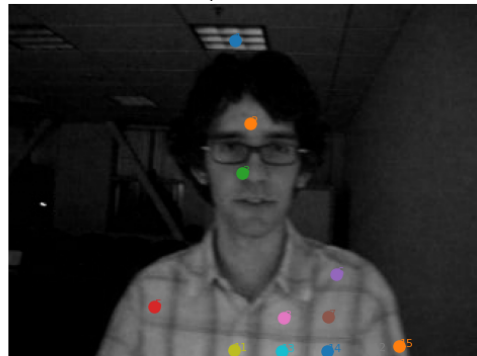
Otsu — distance-based (max_dist=40) — frame 49



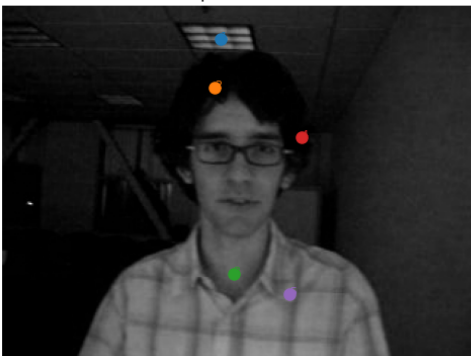
Fixed — descriptor-based — frame 48



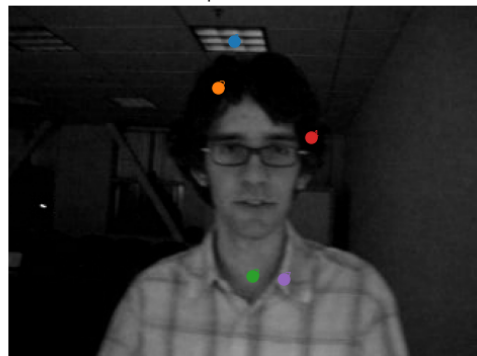
Fixed — descriptor-based — frame 49



Otsu — descriptor-based — frame 48



Otsu — descriptor-based — frame 49



Fixed distance-based matches: 12
Otsu distance-based matches: 5
Fixed descriptor-based matches: 12
Otsu descriptor-based matches: 5

6.4 Comment

As expected a dynamic threshold, like Otsu's threshold, adapts to the light changes drastically reducing the false positive.